UNIVERSIDADE FEDERAL DE MINAS GERAIS
Instituto de Ciências Exatas
Departamento de Ciência da Computação
DCC207 – Algoritmos 2
Prof. Renato Vimieiro

---

# Trabalho Prático 1 – Indexação Geoespacial e Geometria Computacional Híbrida

## Objetivos

Nesse trabalho, serão abordados os aspectos práticos da integração entre estruturas de dados para tratamento de strings e algoritmos de geometria computacional. Especificamente, serão explorados aspectos de implementação de árvores de prefixos (*Tries* ) aplicadas à codificação *Geohash*.

O objetivo secundário é fixar o conteúdo teórico e demonstrar sua aplicabilidade em contextos práticos, tais como sistemas de rastreamento de frotas e serviços de geolocalização em tempo real (ex.: plataformas de entrega e transporte). Entende-se que, ao implementar essas estruturas, o aluno compreenderá com maior profundidade os detalhes relacionados a essas estruturas. Além disso, os alunos terão a oportunidade de trabalhar com bibliotecas para manipulação, processamento e plotagem de mapas e dados geográficos.

---

## Tarefas

Os alunos deverão implementar um motor de busca e monitoramento geoespacial híbrido para visualização e consulta de uma frota de entregadores em operação na cidade de Belo Horizonte. O sistema deverá processar dados de localização geográfica e permitir que o usuário (cliente do app) simule o pedido em restaurantes e identifique os entregadores mais próximos do restaurante. Os resultados deverão ser exibidos como pinos de localização em um mapa interativo, acompanhados de uma tabela descritiva das distâncias e da estimativa de tempo de chegada dos entregadores até o restaurante.

A arquitetura do sistema deve ser dividida em três componentes algorítmicos centrais, que devem ser implementados obrigatoriamente pelos alunos:

1. **Indexação e Busca por Prefixo (*Geohash* e Árvore Trie):**

- Cada coordenada geográfica (latitude, longitude) dos entregadores deverá ser convertida para uma string *Geohash* em Base32 por meio da intercalação binária de intervalos, limitados a 8 níveis.
- Os *Geohashes* gerados deverão ser armazenados em uma *Trie* compacta.
- O sistema deve permitir a busca de entregadores que possam atender a um restaurante específico informado pelo usuário. O sistema deverá codificar a localização do restaurante (*Geohash*), buscar o prefixo correspondente e retornar instantaneamente todos os entregadores contidos naquela região/bairro em tempo $O(m + k)$, onde $m$ é o tamanho do prefixo e $k$ é o número de entregadores encontrados.
- Devido à natureza da grade do *Geohash*, pontos geograficamente muito próximos podem estar em células com prefixos diferentes caso estejam nas bordas de uma divisão. Os alunos deverão adaptar a busca na Trie para cobrir as células adjacentes à do restaurante, eliminando o problema de borda antes da filtragem geométrica final.

2. **Filtragem do Resultado Parcial:**

   - Após recupear os possíveis entregadores, o sistema deverá filtrar somente aqueles que estejam dentro de um raio de atendimento, por exemplo, somente os entregadores que estejam a, no máximo, 2km de distância do restaurante. A definição desse raio deverá estar disponível como parâmetro na interface do sistema.

   - O sistema então deverá plotar os entregadores filtrados em destaque no mapa (e.g. pino vermelho), enquanto os descartados deverão ser plotados sem destaque (pino cinza, por exemplo).

### Interface e Requisitos do Sistema

A implementação deverá ser feita obrigatoriamente em **Python**. O produto final deverá ser uma página interativa que permite a visualização dos dados, busca por endereço/nome do restaurante, ajuste de raio de filtragem e simulação de posição de entregadores.

Ao realizar uma consulta, o mapa deve atualizar dinamicamente os pinos dos entregadores filtrados, bem como a tabela contendo os IDs dos entregadores, suas respectivas distâncias e tempo até o ponto de consulta, além das métricas resumo dos mesmos.

Os alunos deverão criar um repositório no GitHub com todos os arquivos do projeto. Além disso, o repositório deve conter instruções no arquivo `README.md`, com as instruções de execução, bem como um arquivo de dependências (como `requirements.txt`), `Dockerfile` ou outros recursos necessários para garantir a correta execução do projeto.

### Simulador de Geração de Entregadores

Os alunos deverão implementar um gerador de dados sintéticos para o posicionamento da frota, integrado diretamente à interface da ferramenta.

A interface deverá conter um campo numérico onde o usuário informará a quantidade desejada de entregadores ($N$) e um botão de controle (ex.: "Gerar Entregadores"). Ao clicar neste botão, o sistema deverá descartar todos os entregadores armazenados e exibidos anteriormente (limpando as árvores e o mapa) e gerar um novo conjunto de $N$ posições do zero.

Para garantir a validade dos testes, a geração desses dados deverá seguir duas restrições fundamentais:

1. **Distribuição Espacial Baseada em Demanda:** O posicionamento matemático inicial dos entregadores não deverá ser uniforme. Para simular a ociosidade real em aplicativos de entrega, as coordenadas devem ser geradas seguindo uma **distribuição normal** centrada nas coordenadas dos restaurantes (bares da base de dados). O sistema deve distribuir os $N$ entregadores equitativamente entre os centros disponíveis. Como resultado da distribuição normal, o entorno imediato dos restaurantes apresentará alta densidade de entregadores, com o número caindo gradativamente à medida que a distância do centro aumenta.
2. **Restrição Viária (Realismo Geográfico):** A geração puramente matemática da distribuição normal fatalmente posicionará entregadores em locais inacessíveis (como no meio de quarteirões fechados, dentro de edifícios, na lagoa da Pampulha, etc.). Para resolver esse problema, os alunos deverão implementar uma rotina de correção que projete o ponto matemático gerado para a rua transitável mais próxima. Recomenda-se o uso do grafo da malha viária do OpenStreetMap (via bibliotecas como `osmnx`) para mapear a coordenada simulada para a coordenada real de rua mais próxima, garantindo que os pinos no mapa apareçam exclusivamente sobre as vias da cidade.

### Tecnologias Recomendadas

A construção do mapa interativo poderá ser feita com o auxílio da biblioteca `dash-leaflet` ou `plotly` / `dash`. Essas bibliotecas possibilitam a integração entre mapas interativos baseados em OpenStreetMap e Python.

### Organização e qualidade da solução

A qualidade do produto final também será levada em consideração como critério de avaliação. Implementações corretas, mas com aspecto (estética da solução) ruins, serão penalizadas. Não é necessária a implementação de uma interface extremamente sofisticada, contudo, espera-se que ela seja esteticamente organizada, acessível, mais próxima à interface de um produto final.

### Relatório Final

Os alunos deverão preparar um relatório em que descrevem sua implementação bem como o problema abordado no trabalho. Deverão ser descritos todos os passos da implementação, destacando as decisões de projeto tomadas (ex.: estrutura de nós da Trie, geração dos dados sintéticos, correção da posição dos entregadores). O relatório deve conter uma **seção experimental** mostrando a execução de algumas consultas realizadas, incluindo prints da interface que mostrem o resultado. Os alunos deverão incluir também o tempo de resposta (runtime) para cada consulta.

O nível de elaboração do texto, clareza e qualidade das descrições e gráficos serão critérios centrais de avaliação. Em outras palavras, o mesmo cuidado com a implementação deverá ser observado no relatório produzido. O relatório deverá ser publicado junto com o código em repositório aberto no GitHub (formato PDF ou `README.md` estruturado).

O trabalho poderá ser feito em grupos de até três alunos. Recomenda-se fortemente que o trabalho seja realizado em grupo.

O uso de qualquer biblioteca adicional para as estruturas de dados centrais (como uso de bibliotecas prontas para *trie*) é **estritamente proibido**; o código das estruturas e algoritmos deve ser de autoria dos alunos. O uso de bibliotecas de terceiros para interface gráfica, mapas ou suporte a E/S (como `pandas`) deverá ser discutido com o professor ou o monitor.

---

## O que entregar?

Deverá ser entregue um repositório no GitHub contendo todos os arquivos criados na implementação da ferramenta, scripts de geração/teste e o relatório. O link para o repositório deverá ser postado no Moodle. O repositório deverá ser mantido privado até 3 dias após a data de entrega. Então, o repositório deverá ser tornado público. Caso o repositório não esteja aberto ou o link postado não funcione a partir do 4º dia após a data de entrega, o trabalho será considerado não entregue e receberá nota nula.

---

## Política de Plágio

Os alunos podem, e devem, discutir soluções sempre que necessário. Dito isso, há uma diferença bem grande entre implementação de soluções similares e cópia integral de ideias. Trabalhos copiados na íntegra ou em partes de outros alunos e/ou da internet serão prontamente anulados. Caso haja dois trabalhos copiados por alunos/grupos diferentes, ambos serão anulados. O uso de ferramentas de IA para implementação do trabalho será considerado como plágio.

---

## Datas

**Entrega:** 18/10/2026 às 23h59

---

## Política de atraso

Haverá tolerância de 30min na entrega dos trabalhos. Submissões feitas depois do intervalo de tolerância serão penalizadas, incluindo mudanças no repositório.

- **Atraso de 1 dia:** 30% de penalização
- **Atraso de 2 dias:** 50% de penalização
- **Atraso de 3+ dias:** Não aceito

Serão considerados atrasos de 1 dia aqueles feitos após as 0h30 do dia seguinte à entrega. A partir daí será contado o número de dias passados desde a data de entrega.